map()

Previously, to convert from a 10-bit value to an 8-bit one, we simply divided the analog reading by 4 and applied that value to our analog output. An alternative method would be to use the `map()` function, which appropriately enough, is one of the more useful functions for mapping sensor values. There are several ways that it can be used, but let's look at its syntax first.

```
map(value, fromLow, fromHigh, toLow, toHigh)
```

The way it works is that we pass the function a total of five values, beginning by first defining the value that we want to map. This is most often a variable, but could also be a call to another function. It should be a whole number integer value, as floating-point math is ignored after the decimal point. The second and third values are the low and high values that we are beginning with expressed as constants or variables. This is where the Analog Serial Monitor comes in handy, allowing us to better plan for the values coming from our sensors. Finally, the fourth and fifth values are the new low and high values that we want to map the old values to. A good example of this is using `map()` to convert a 10-bit reading to an 8-bit value, as follows:

```
analogValue = map(analogValue, 0, 1023, 0, 255);
```

This statement will take the value from the variable `analogValue` with expected values that begin at 0 and go up to a maximum of 1023, and map these values to the new values of 0 to 255. This line effectively converts a range of values from 0 to1023 to 0 to 255. That's pretty handy for driving an analog output with an analog input.

Another thing that comes up frequently is the need to reverse a range of values. For example, using the code in Listing 6-3, the LED gets brighter when the light in the room is brighter. What if we wanted the LED to get brighter when it actually gets darker in the room? To do this we would need to reverse the range of values so that instead of outputting 0–255 we would flip it to 255–0. The `map()` function can be used to do exactly that.

```
analogValue = map(analogValue, 0, 1023, 255, 0);
```

By flipping the last two values, we are functionally flipping the values that we plan to use from the analog input. We can also use the `map()` function to set up ranges or thresholds of values that can trigger different events. This works well with the switch statement, just like the following code fragment:

```
analogValue = map(analogRead(analogPin), 0, 1023, 1, 3);
  switch (analogValue) {
    case 1:
    ;
    case 2:
    ;
    case 3:
    ;
  }
```

In this fragment, we are taking an analog reading and boiling it down to three possible states. Think of it as near, not too far, and far away; cold, warm, and too hot; dark, just right, and bright. What we do with those three states, I leave up to you.